Lab 10 — Commented answers

Each answer follows the same pattern: the answer, the mechanism, the nuance or pitfall, an example you can verify at the terminal.

Question 1 — 40 GB in `userdata/`

Answer. It is the container's **log file** (`ctr.log`, `k8s-file` driver): a chatty application that has been running for weeks, with no rotation. `podman system df` counts images, writable layers and volumes, not logs: in its eyes they are neither "reclaimable" nor "removable". Immediate fix: recreate the container (the logs go with it) or truncate the file (`: > ctr.log`, acceptable in an emergency). Lasting fix: cap the size (`--log-opt max-size=10m`, or `log_size_max` in `containers.conf`), ship the logs to a central platform, and lower the application's log level.

Why. The engine captures stdout into a file that has no limit by default; only a rotation or a maximum size stops it.

Nuance. With the `journald` driver (the default on an Ubuntu with systemd), it is the system journal that grows, capped by its own configuration (`SystemMaxUse`).

Example.

```
podman inspect api --format '{{.HostConfig.LogConfig.Type}} {{.HostConfig.LogConfig.Path}}'
du -sh $(podman inspect api --format '{{.HostConfig.LogConfig.Path}}')
```

Question 2 — `Exited (137)` without `OutOfMemoryError`

Answer. With `MaxRAMPercentage=90`, the Java heap can reach 230 MB on a 256 MB limit; the rest of the JVM (metaspace, threads, JIT code, buffers) exceeds the remaining 26 MB. The cgroup's **total** memory hits `memory.max`, the kernel triggers the OOM killer and kills `java` with `SIGKILL`: code `137`, no Java exception (the JVM never ran out of heap; the container ran out of memory). Confirmation: `podman inspect api --format '{{.State.OOMKilled}}'` → `true`, and `podman events --filter event=oom`. Fixes: lower `MaxRAMPercentage` to 50-75 %, or raise `--memory`.

Why. The cgroup limit applies to the **whole** process; the heap is only part of it. `OutOfMemoryError` only occurs when the **heap** is full, which never happens when the kernel strikes first.

Nuance. `-XX:+ExitOnOutOfMemoryError` and a `--restart` mask the symptom without fixing it. Sizing means measuring: `podman stats` during a load test.

Example.

```
podman inspect api --format 'oom={{.State.OOMKilled}} exit={{.State.ExitCode}}'   # oom=true exit=137
podman exec api sh -c 'java -XX:MaxRAMPercentage=90 -XX:+PrintFlagsFinal -version 2>/dev/null | grep " MaxHeapSize"'
```

Question 3 — `stats`, `top`, `inspect`

Answer. `podman stats` reads the **cgroup counters** in real time: CPU %, memory used / limit, network, I/O. `podman top` lists the container's **processes** (UID, PID, command), like `ps` seen from inside. `podman inspect` describes the **configuration and state**: what was requested (limits,

variables, mounts), not what is consumed. Without `--memory`, the cgroup has no ceiling (`memory.max = max`) and `stats` shows the **machine's** memory as the limit: 32 GB does not mean "reserved", but "nothing stops this container before that".

Why. Three sources: the kernel (cgroups, `/proc`) for `stats` and `top`, Podman's database for `inspect`.

Nuance. `stats` under rootless requires cgroup delegation; `top` accepts columns (`podman top api user pid args`) but not the full `ps` syntax.

Example.

```
podman stats --no-stream --format 'table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}'
podman inspect api --format 'mem={{.HostConfig.Memory}}'   # 0 = no limit
```

Question 4 — The vanished `HEALTHCHECK`

Answer. Buildah builds an image in the **OCI** format by default, whose specification has no field for the healthcheck: the instruction is ignored, with a single warning lost in the build output (`HEALTHCHECK is not supported for OCI image format and will be ignored`). Check: `podman image inspect api:1.1 --format '{{.Config.Healthcheck}}'` → `<nil>`. Fix: `podman build --format docker -t api:1.1 .`, after which the same inspect shows `[CMD-SHELL wget …]` and `podman ps` displays `(healthy)`. Docker builds in the Docker format by default: the healthcheck is always kept there.

Why. `HEALTHCHECK` is a Docker extension of the image format, never adopted by the OCI standard (which leaves that role to the orchestrator).

Nuance. An OCI image remains the right target for Kubernetes, which ignores the `HEALTHCHECK` anyway and uses its own probes. The docker format only matters for Podman/Docker/Compose.

Example.

```
podman build --format docker -q -t api:1.1 . && podman image inspect api:1.1 --format
'{{.Config.Healthcheck.Test}}'
```

Question 5 — `unhealthy`, and then?

Answer. With Podman alone: **nothing**. The state turns `unhealthy` in `podman ps`, a `health_status` event is emitted, the container keeps running and answering badly. Docker does the same: the healthcheck is an **indicator**, not a fixer; only Swarm or Compose with `depends_on` act on it. Podman adds `--health-on-failure=kill|restart|stop`: with `kill` and `--restart=always`, the container is killed then relaunched. The orchestrator remains the real answer because it combines the probe with **removing traffic** (readiness), **replacement** (liveness) and **relocation** if the machine is the cause.

Why. Deciding to kill a service is a steering decision, which the container engine deliberately leaves to a higher layer.

Nuance. A badly tuned liveness probe kills a slow service in a loop: the probe must test **liveness**, not load.

Example.

```
podman run -d --name api --health-on-failure=kill --restart=always api:1.1
podman events --stream=false --since 10m --filter event=health_status | tail -2
```

Question 6 — The hardened container

Answer. `uid=1000(…) gid=1000(…)` (the image has `USER 1000:1000`, `--user 1000` confirms it); `touch: /app/x: Read-only file system`; `touch: /tmp/x: Read-only file system` **as well**, since no `--tmpfs /tmp` was given; `java -version` works (reading needs nothing). For a real Spring Boot API: add `--tmpfs /tmp` (Tomcat creates its working directory there, the JVM writes `hsperfdata` there), and if the application writes files (uploads, cache), a dedicated volume. Nothing else: an API needs no capability at all.

Why. `--read-only` locks the whole image, `/tmp` included; legitimate writes must have an explicit mount.

Nuance. `--cap-drop ALL` with `--user 1000` is almost redundant (a non-root user already lacks the capabilities), but protects against a setuid binary in the image — which `no-new-privileges` locks down too.

Example.

```
podman run --rm --read-only --tmpfs /tmp --cap-drop ALL --user 1000 --security-opt no-new-
privileges --entrypoint sh api:1.0 -c 'touch /tmp/x && echo OK'
```

Question 7 — Root without powers

Answer. In the first case, root holds the container's default capabilities, including `CAP_CHOWN`; `chown` succeeds. With `--cap-drop ALL`, the process is still UID 0, but with **no** capability at all: the kernel refuses `chown` (`Operation not permitted`), as it would for an ordinary user. The example shows that, on Linux, "root" is just a number; **privileges** are capabilities attached to the process, which can be removed one by one. An attacker who becomes root in a `--cap-drop ALL` container only has the rights of a powerless user on a filesystem they cannot modify.

Why. Since Linux 2.2, the kernel's privilege checks test capabilities, not the UID; UID 0 is just one way to obtain the full set.

Nuance. Under rootless, the container's capabilities only hold inside the **user namespace**: even `CAP_SYS_ADMIN` gives no power over the host there. Two layers of protection.

Example.

```
podman run --rm --entrypoint sh alpine -c 'chown nobody /etc/hostname && echo OK'          # OK
podman run --rm --cap-drop ALL --entrypoint sh alpine -c 'chown nobody /etc/hostname'      #
Operation not permitted
```

Question 8 — `latest` in production

Answer. Three incidents: (1) **two servers, two versions** — one `pull` at 10:00, another at 10:05 after a new build, and nobody knows which runs where; (2) **no rollback** — `latest` has been overwritten, the previous image has no name any more; (3) **non-reproducible deployment** — a `pull` in staging and one in production do not yield the same content, the test proves nothing. Strategy: `api:1.4.2` + `api:1.4` + `api:1` + `api:git-<sha>`, never deploy `latest`. Deploying **by**

digest additionally guarantees that even a **rewritten** version tag (by mistake, or by an attacker of the registry) does not change what runs: the digest is the fingerprint of the content, a tag is only a name.

Why. A tag is a movable pointer stored on the registry side; a digest is computed from the layers.

Nuance. Floating tags (`1.4`, `1`) are useful for **consumers** (a base image), not for deploying **your** application.

Example.

```
podman image inspect api:1.0 --format '{{.Digest}}'
podman pull localhost:5000/api@sha256:52b82ed9…   # immutable, whatever happens to the tag
```

Question 9 — HTTPS expected

Answer. Podman speaks **TLS** by default to every registry, including `localhost:5000`; `registry:2` without a certificate answers in plain HTTP, hence `server gave HTTP response to HTTPS client`. Local test: `--tls-verify=false` on `push` and `pull`. Company: a valid **certificate** on the registry (and the CA in `/etc/containers/certs.d/<registry>/ca.crt` if it is internal). In between, `[[registry]] location="…" insecure=true` in `registries.conf` declares the exception once and for all. The former must never become the latter because a plain-text registry can be **intercepted and replaced**: an image substituted on the network path compromises the whole deployment chain.

Why. The registry distributes executable code; TLS guarantees its origin and integrity in transit.

Nuance. Docker refuses the same way and is configured through `insecure-registries` in `daemon.json` — a daemon setting, hence an administrator's.

Example.

```
podman push --tls-verify=false localhost:5000/api:1.0 && curl -s localhost:5000/v2/_catalog
```

Question 10 — Same `IMAGE ID` after the round trip

Answer. The registry received the image's **layers** (blobs), **configuration** and **manifest**, each identified by its hash; at `pull`, Podman verified every hash and rebuilt exactly the same image. The `IMAGE ID` is the hash of the configuration: identical content, identical ID — the name `localhost:5000/api:1.0` is just one more tag on the same local image. `--format '{{.Digest}}'` gives the hash of the **manifest**, the one the registry uses: that is what you reference for an immutable deployment, and what changes if a layer changes.

Why. Storage is **content-addressed**: two paths to the same content lead to the same object.

Nuance. The same content pushed to two registries can have two manifest digests if the format differs (OCI versus Docker); the configuration ID stays identical.

Example.

```
podman images --format '{{.Repository}}:{{.Tag}} {{.ID}}' | grep api     # two names, one ID
curl -s localhost:5000/v2/api/manifests/1.0 -H 'Accept:
application/vnd.oci.image.manifest.v1+json' | head -c 200
```

Question 11 — `system prune -a --volumes -f` in the middle of CI

Answer. Removed: every **stopped** container, unused networks, **every** image without a running container (`-a`: base images, `api:1.0`, build cache), and every volume no **existing** container uses. Not removed: running containers, their images and their volumes — the test stack survives... until its `down`, after which its volumes become orphans. Next build: re-download of base images (minutes, Docker Hub quota), rebuild without cache, and loss of the data volumes shared between jobs. Correct sequence: `podman system df`, `podman image prune` (dangling only), `podman container prune` if the stopped containers are known, **named** deletion of volumes after reading `volume ls --filter dangling=true`.

Why. `prune` reasons in "referenced by an existing container", not in "useful tomorrow".

Nuance. On an ephemeral CI agent (recreated for each job), `prune -a` is on the contrary the norm; the problem lies with the **persistent** agent where the cache is precious.

Example.

```
podman system df && podman volume ls --filter dangling=true && podman image prune -f
```

Question 12 — The Quadlet

Answer. `daemon-reload` makes the Quadlet generator read the `api.container` file, which produces an `api.service` unit with an `ExecStart=podman run … localhost/api:1.0` and the translated options (`-p 18080:8080`, `--env-file`). `enable --now` starts the container and registers it in `default.target`: at every start of the user session, systemd relaunches `podman run`. `--restart=always` fails after `wsl --shutdown` because no Podman process survives to apply it; here, it is **systemd** that the system relaunches, and systemd relaunches Podman. On WSL, you need `loginctl enable-linger $USER` so that the `--user` session exists without a login, and the distribution must be started (an open `wsl` or a scheduled task).

Why. A systemd service has a permanent supervisor; a `--restart` container has none without a daemon.

Nuance. Quadlet generates **disposable** containers (`podman run --rm --replace`): data must live in volumes, ideally declared with a `.volume` Quadlet.

Example.

```
systemctl --user daemon-reload && systemctl --user enable --now api && systemctl --user status
api --no-pager | head -3
journalctl --user -u api -n 5 --no-pager
```

Question 13 — The pipeline

Answer. (1) **Build** — `podman build --format docker -t api:1.4.2 -t api:git-a1b2c3d .` (labs 04-05); fails if the Dockerfile or the tests of the `test` stage fail. (2) **Scan** — Trivy/Grype on the image (lab 05); fails on a critical CVE. (3) **Push** — `podman push` to the private registry, with TLS (labs 02, 10); fails if the registry refuses (rights, quota). (4) **Staging** — `podman compose -f compose.yaml -f compose.staging.yaml up -d` on the image **by digest**, integration tests, `down -v` (lab 09); fails if a test or a healthcheck does not pass. (5) **Production** — the orchestrator deploys the **same digest** (labs 08, 11+), with the limits, the `securityContext` and the probes written in the manifests. The scan sits before the push (you do not distribute a vulnerable image); the `HEALTHCHECK` is in the

image from the build; resource limits are in the deployment files (Compose or manifests), not in the image; the digest is computed at build, recorded at push, and referenced from staging onwards.

Why. Each step produces a verifiable, immutable artefact for the next: image, scan report, digest, test result.

Nuance. The pipeline is the same with Docker; only the build/push commands and the image format change.

Example.

```
DIGEST=$(podman image inspect api:1.4.2 --format '{{.Digest}}') && echo "deploy:
registry.example.com/api@$DIGEST"
```